Derivation of Weight Update Formulas for a Neural Network

Cross-Entropy Loss with Softmax Output (Multi-Class Classification)

---

1. Introduction

Artificial Neural Networks (ANNs) are widely used for multi-class classification problems. Training such networks requires minimizing a loss function that measures the discrepancy between predicted outputs and true labels. While the **Mean Squared Error (MSE)** loss can be used, it is not optimal for classification tasks. Instead, the **cross-entropy loss** combined with the **softmax activation function** in the output layer is the standard and mathematically well-justified choice.

This report derives the **weight update formulas (Δw)** for both the **output layer** and **hidden layers** of a feedforward neural network trained using **backpropagation**, when **softmax activation** and **cross-entropy loss** are used. The derivation closely mirrors the MSE-based approach but highlights the key simplification that arises in the softmax–cross-entropy combination.

---

2. Network Architecture and Notation

We consider a standard feedforward neural network consisting of:

- An **input layer**
- One or more **hidden layers**
- An **output layer** with softmax activation

Notation

- i: index of neurons in the previous layer
- j: index of neurons in a hidden layer
- k: index of neurons in the output layer

Forward Pass Variables

- x_i: input to the network
- w_{ij}: weight from neuron i to neuron j
- w_{jk}: weight from neuron j to neuron k
- z_k: pre-activation (net input) of output neuron k
- a_k: output of neuron k after softmax
- t_k: target (true) label for class k

---

3. Softmax Activation Function

The softmax function converts raw scores into probabilities:

$$a_k = \text{softmax}(z_k) = \frac{e^{z_k}}{\sum_m e^{z_m}}$$

Properties:

- $0 \leq a_k \leq 1$
- $\sum_k a_k = 1$

This makes softmax ideal for **multi-class classification**.

---

4. Cross-Entropy Loss Function

For a single training example, the cross-entropy loss is defined as:

$$\text{L} = -\sum_k t_k \ln(a_k)$$

where:

- $t_k = 1$ for the correct class
- $t_k = 0$ for all other classes

---

5. Backpropagation: Output Layer Derivation

5.1 Objective

We aim to compute the gradient:

$$\frac{\partial \text{L}}{\partial w_{jk}}$$

using the chain rule.

5.2 Chain Rule Expansion

$$\frac{\partial \text{L}}{\partial w_{jk}} = \frac{\partial \text{L}}{\partial a_k} \cdot \frac{\partial a_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{jk}}$$

5.3 Individual Derivatives

1. Loss derivative:

$$\frac{\partial \mathrm{L}}{\partial a_k} = -\frac{t_k}{a_k}$$

2. Softmax + Cross-Entropy Simplification

A key result:

$$\frac{\partial \mathrm{L}}{\partial z_k} = a_k - t_k$$

This simplification occurs because the Jacobian of softmax cancels terms from the cross-entropy gradient.

3. Net input derivative:

$$\frac{\partial z_k}{\partial w_{jk}} = a_j$$

5.4 Final Gradient (Output Layer)

$$\frac{\partial \mathrm{L}}{\partial w_{jk}} = (a_k - t_k)a_j$$

5.5 Weight Update Rule (Output Layer)

Using gradient descent with learning rate η:

$$\boxed{\Delta w_{jk} = -\eta(a_k - t_k)a_j}$$

---

6. Backpropagation: Hidden Layer Derivation

6.1 Objective

Compute the gradient for hidden layer weights:

$$\frac{\partial \mathrm{L}}{\partial w_{ij}}$$

6.2 Chain Rule Expansion

$$\frac{\partial L}{\partial w_{ij}} = \sum_k \frac{\partial L}{\partial z_k} \cdot \frac{\partial z_k}{\partial a_j} \cdot \frac{\partial a_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ij}}$$

6.3 Individual Derivatives

- Output error term:

$$\frac{\partial L}{\partial z_k} = a_k - t_k$$

- Weighted contribution:

$$\frac{\partial z_k}{\partial a_j} = w_{jk}$$

- Activation derivative (e.g., sigmoid):

$$\frac{\partial a_j}{\partial z_j} = f'(z_j)$$

- Net input derivative:

$$\frac{\partial z_j}{\partial w_{ij}} = a_i$$

6.4 Hidden Layer Error Term

Define the hidden neuron error:

$$\delta_j = f'(z_j) \sum_k (a_k - t_k) w_{jk}$$

6.5 Final Gradient (Hidden Layer)

$$\frac{\partial L}{\partial w_{ij}} = \delta_j a_i$$

6.6 Weight Update Rule (Hidden Layer)

$$\boxed{\Delta w_{ij} = -\eta \, \delta_j \, a_i}$$

7. Comparison with MSE Loss

Aspect	MSE	Cross-Entropy + Softmax
Output error	$(a_k - t_k)f'(z_k)$	$a_k - t_k$
Gradient complexity	Higher	Simplified
Numerical stability	Lower	Higher
Classification performance	Suboptimal	Optimal

The elimination of $f'(z_k)$ in the output layer is the key advantage.

---

8. Conclusion

This report presented a complete derivation of the **weight update formulas** for neural networks trained using **cross-entropy loss with softmax activation**. By applying the chain rule step-by-step, we demonstrated that:

- The output layer gradient simplifies to $a_k - t_k$
- The hidden layer update retains the familiar backpropagation structure
- The softmax–cross-entropy combination is mathematically elegant and computationally efficient

These properties explain why this loss-activation pairing is the **standard choice for modern multi-class neural network training**.